BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC ĐẠI NAM

BÀI TẬP LỚN

TÊN HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI: GIẢI THUẬT ĐỆ QUY

Giáo viên hướng dẫn: Trần Quý Nam

Sinh viên thực hiện:

Stt	Mã sv	Họ và tên	Lớp
01	1871020471	Phạm Anh Quân	CNTT18-02
02	1871020538	Nguyễn Chung Thành	CNTT 18-02
03	1871020570	Đặng Quang Tiến	CNTT 18-02

Hà Nội, năm 2024

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC ĐẠI NAM

BÀI TẬP LỚN

TÊN HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI: giải thuật đệ quy

STT	Mã Sinh Viên	Họ và Tên	Ngày Sinh	Điểm	
				Bằng Số	Bằng Chữ
1	1871020471	Phạm Anh Quân	09/07/2006		
2	1871020538	Nguyễn Chung Thành	CNTT 18-02		
3	1871020570	Đặng Quang Tiến	CNTT 18-02		

CÁN BỘ CHẤM THI 1 **CÁN BỘ CHẤM THI 2**

Trần Quý Nam

Hà Nội, năm 2024

LỜI NÓI ĐẦU

(Nếu có)

Giải thuật đệ quy là một trong những thuật toán trong nâng cao trong C++. Thuật toán này giúp ta đi sâu vào trong cấu trúc và am hiểu rõ hơn về ngôn ngữ lập trình này mà một nhà lập trình từ mới bắt đầu đến chuyên sâu đều cần phải nắm bắt rõ để các thuật toán của mình đi chuyên sâu hơn.

Trong 3 chương ta sẽ làm rõ hơn về cách thức hoạt động đến cách thức sử dụng lẫn ứng dụng của thuật toán này trong ngôn ngữ lập trình C++.

Chương 1: ta sẽ tìm hiểu về khái niệm, nguồn gốc, mục đích, và ý nghĩa của thuật toán này đối với người lập trình cũng như là ngôn ngữ C++.

Chương 2: ta sẽ tìm hiểu sâu thêm về giải thuật đệ quy từ cách thức hoạt động của thuật toán, trình tự sử lý công việc, cách thức các dòng lệnh liên kết với nhau, giải thích cách thức hoạt động của code, trình tự in ra màn hình.

Chương 3: ứng dụng của giải thuật đệ quy trong các chương trình trong C++

Kết luận về giải thuật đệ quy. Phân tích điểm mạnh điểm yếu, phương hướng phát triển của đề tài.

Đối tượng nghiên cứu: hàm đệ quy và những hàm liên quan

Phạm vi nghiên cứu: chỉ nằm trong đệ huy và một số hàm liên quan trong ngôn ngữ C++, sử dụng ngôn ngữ thường cũng như code để mô tả các ý để tường thuật lại cách hoạt động một cách đơn giản dễ hiểu nhất.

Phương pháp nghiên cứu: đọc, thu thập từ giáo trình, sách, cùng một số tài liệu liên quan

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ GIẢI THUẬT ĐỆ QUY ..... **Error! Bookmark not defined.**

1.1 giải thuật đệ quy ..... **Error! Bookmark not defined.**

1.2 Các yếu tố của đệ quy ..... **Error! Bookmark not defined.**

1.3 Ý nghĩa của hàm đệ quy trong ngôn ngữ C++, cũng như với người lập trình viên **Error! Bookmark not defined.**

CHƯƠNG 2: PHÂN TÍCH CHUYÊN SÂU VỀ HÀM ĐỆ QUY ..... **Error! Bookmark not defined.**

2.1 Các dạng đệ quy ..... **Error! Bookmark not defined.**

2.2 Cấu tạo của hàm đệ quy ..... **Error! Bookmark not defined.**

2.3 Các thành phần của một hàm đệ quy ..... **Error! Bookmark not defined.**

2.5 Cơ chế hoạt động của một hàm đệ quy ..... **Error! Bookmark not defined.**

2.7 So sánh đệ quy với các phương pháp lập trình khác ..... **Error! Bookmark not defined.**

CHƯƠNG 3: ĐỆ QUY TRONG THỰC TẾ ..... **Error! Bookmark not defined.**

3.1 MỘT SỐ BÀI TOÁN ĐỆ QUY ..... **Error! Bookmark not defined.**

3.1.1 bài toán sinh các dãy nhị phân ..... **Error! Bookmark not defined.**

3.1.2 Sinh tổ hợp (tập hợp con) ..... **Error! Bookmark not defined.**

3.2 những ứng dụng thực tế của đệ quy ..... **Error! Bookmark not defined.**

3.3 Những lỗi thường gặp khi dung đệ quy ..... **Error! Bookmark not defined.**

KẾT LUẬN ..... **Error! Bookmark not defined.**

Phân công công việc các thành viên trong nhóm ..... **Error! Bookmark not defined.**

DANH MỤC TÀI LIỆU THAM KHẢO..... **Error! Bookmark not defined.**

MỤC LỤC HÌNH ẢNH

ảnh 2.1 cấu trúc của đệ quy trực tiếp 12

ảnh 2.2 ví dụ minh họa về đệ quy gián tiếp 13

ảnh 2.3 ví dụ minh họa về đệ quy gián tiếp 14

ảnh 2.4 ví dụ minh họa về đệ quy đuôi 14

ảnh 2.5 cách thức hoạt động của đệ quy đuôi 15

ảnh 2.6 cách thức hoạt động của hàm đệ quy 16

- ảnh 2.7. ví dụ về đệ quy tuyến tính 18
- ảnh 2.8 cách thức hoạt động của ví dụ trên 19

ảnh 2.9 ví dụ minh họa về đệ quy nhị phân 21

ảnh 2.10 ví dụ minh họa về cơ chế hoạt động của hàm đệ quy 23

ảnh 2.11 cách thức hoạt độn theo sơ đồ 24

3.1.1 bài toán sinh các dãy nhị phân 28

ảnh 3.1 phân tích hàm đệ quy của bài toán sinh các dãy nhị phân 28

3.2 code hàm main của bài toán sinh các dãy nhị phân 29

3.3 code toàn bộ bài toán sinh các dãy nhị phân 29

ảnh 3.4 kết quả của bài toán sinh các dãy nhị phân 30

3.1.2 Sinh tổ hợp (tập hợp con) 30

ảnh 3.5 code hàm đệ của bài toán sinh tổ hợp 31

3.6 code hàm main của bài toán sinh tổ hợp 32

3.7 kết quả của bài toán sinh tổ hợp 32

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ GIẢI THUẬT ĐỆ QUY

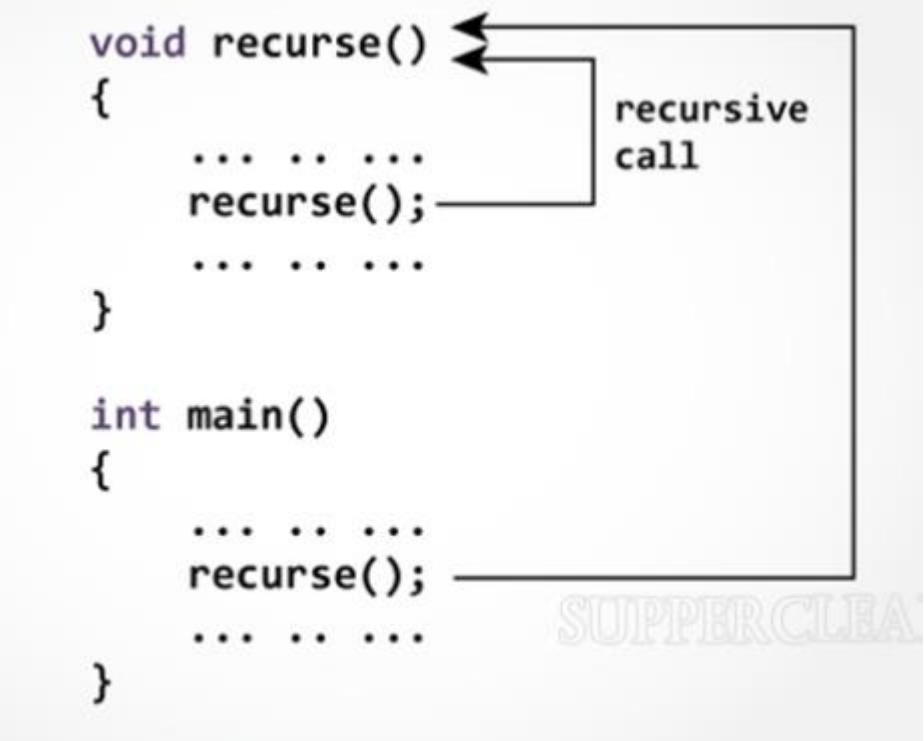
1.1giải thuật đệ quy

Trong khuôn khổ của Tin học, việc giải bài toán có nghĩa trao được cho máy tính cách giải. Để giao bài toán cho máy tính, ta cần hướng dẫn cho máy tính các thao tác mà về nguyên tắc máy có thể thực hiện được. Một cách giải bài toán như vậy được các thao tác đơn giản được sắp xếp theo một cách trình tự xác định sao cho sau khi thực hiện dãy thao tác đó, từ Input của bài toán, ta nhận được Output cần tìm.

Trong định nghĩa nêu trên, thao tác đơn giản được hiểu là thao tác mà máy tính có thể thực hiện được. Ví dụ, các phép toán số học, các phép so sánh hai giá trị số là các thao tác đơn giản. Theo đó, việc giải bài toán theo quan niệm của Tin học là quá trình thực hiện một dãy hữu hạn các thao tác đơn giản để có thể từ Input dẫn ra Output một cách tường minh. Chính quan niệm về việc giải bài toán như vậy là cốt lõi để ta có thể chuyển giao công việc đó cho máy tính.

Giải thuật đệ quy Đệ quy là một khái niệm cơ bản trong toán học và khoa học máy tính. Một đối tượng là đệ quy nếu nó hoặc một phần của nó được định nghĩa thông qua khái niệm chính nó.

Có dạng



ảnh 1.1 cách thức hoạt động của giải hệ đệ quy

1.2 Các yếu tố của đệ quy

Đệ quy là một kỹ thuật mạnh mẽ trong lập trình, cho phép một hàm tự gọi chính nó để giải quyết các bài toán phức tạp. Để một hàm đệ quy hoạt động chính xác, nó cần có các yếu tố sau:

1. Trường hợp cơ bản (Base Case):

-Đây là điều kiện dừng của đệ quy. Nếu không có trường hợp cơ bản, hoặc trường hợp cơ bản không bao giờ đạt được, hàm đệ quy sẽ gọi chính nó vô hạn lần, dẫn đến lỗi tràn ngăn xếp (stack overflow).

-Trường hợp cơ bản xác định khi nào hàm đệ quy sẽ dừng lại và trả về một giá trị cụ thể.

2. Bước đệ quy (Recursive Step):

-Đây là phần mà hàm tự gọi chính nó.

-Mỗi lần gọi đệ quy, bài toán được chia nhỏ thành các bài toán con tương tự, nhưng có kích thước nhỏ hơn.

-Bước đệ quy phải đảm bảo rằng bài toán con sẽ tiến gần hơn đến trường hợp cơ bản.

3. Sự thay đổi trạng thái (State Change):

-Trong mỗi lần gọi đệ quy, trạng thái của bài toán phải thay đổi theo cách nào đó để tiến gần hơn đến trường hợp cơ bản.

-Điều này thường liên quan đến việc thay đổi giá trị của các tham số đầu vào của hàm.

Ví dụ minh họa:

Xét hàm tính giai thừa của một số nguyên dương n:

Trường hợp cơ bản: n == 0, hàm trả về 1.

Bước đệ quy: return n * giai_thua(n - 1), hàm gọi chính nó với n - 1.

Trường hợp cơ bản: n == 0, hàm trả về 1.

Bước đệ quy: return n * giai_thua(n - 1), hàm gọi chính nó với n - 1.

Sự thay đổi trạng thái: n giảm đi 1 trong mỗi lần gọi đệ quy.

1.3 Ý nghĩa của hàm đệ quy trong ngôn ngữ C++, cũng như với người lập trình viên

Ý nghĩa của hàm đệ quy:

- **Định nghĩa:**

  Hàm đệ quy là một hàm tự gọi lại chính nó trong quá trình thực thi.

  Điều này cho phép giải quyết các bài toán phức tạp bằng cách chia nhỏ chúng thành các bài toán con tương tự nhưng đơn giản hơn.

- **Cấu trúc:**

Một hàm đệ quy thường có hai thành phần chính:

Trường hợp cơ sở (base case): Điều kiện dừng của đệ quy, giúp tránh việc hàm tự gọi vô hạn lần.

Trường hợp đệ quy (recursive case): Phần hàm gọi lại chính nó, với tham số được thay đổi để tiến gần hơn đến trường hợp cơ sở.

- trong ngôn ngữ C++, giải thuật đệ quy nắm vai trò lớn trong cấu trúc hoạt động của của ngôn ngữ này
- **Biểu diễn rõ ràng**: Giải thuật đệ quy giúp mã nguồn trở nên ngắn gọn và dễ hiểu hơn khi làm việc với các bài toán có bản chất lặp hoặc phân rã, ví dụ như tính giai thừa, Fibonacci, hoặc duyệt cây.
-
- **Giải quyết bài toán chia để trị**: Đệ quy là nền tảng cho phương pháp chia để trị (divide and conquer), giúp chia bài toán lớn thành các bài toán con nhỏ hơn, như thuật toán sắp xếp nhanh (QuickSort) hoặc tìm kiếm nhị phân.
-
- **Hỗ trợ xử lý cấu trúc dữ liệu phức tạp**: Đệ quy là cách tiếp cận tự nhiên để làm việc với các cấu trúc như cây và đồ thị, ví dụ như duyệt cây theo cách tiền tự (pre-order), trung tự (in-order), hoặc hậu tự (post-order).
-
- **Tái sử dụng lời giải**: Kết hợp đệ quy với ghi nhớ (memoization) có thể giảm thời gian thực hiện bằng cách lưu trữ các kết quả đã tính toán, áp dụng trong kỹ thuật lập trình động.
-
- **Tính linh hoạt và tự nhiên**: Giải thuật đệ quy cho phép lập trình viên định nghĩa các hàm bằng cách sử dụng chính hàm đó trong thân hàm, điều này rất phù hợp với nhiều bài toán đệ quy tự nhiên trong toán học và lập trình.
-

- Đối với lập trình viên: Giải thuật đệ quy mang lại nhiều ý nghĩa quan trọng đối với lập trình viên trong việc phát triển phần mềm và giải quyết bài toán, cụ thể:

  Hiểu sâu về bài toán: Khi lập trình viên áp dụng đệ quy, họ cần hiểu rõ cấu trúc bài toán, cách chia nhỏ bài toán lớn thành các bài toán con. Điều này giúp tăng khả năng phân tích và tư duy logic.

  Biểu diễn tự nhiên: Đệ quy cho phép lập trình viên biểu diễn các bài toán phức tạp một cách đơn giản và tự nhiên hơn, đặc biệt với những bài toán có cấu trúc đệ quy, như cây, đồ thị, hoặc dãy số.

  Kỹ năng tối ưu hóa: Sử dụng đệ quy thường đi kèm với tối ưu hóa như ghi nhớ (memoization) hoặc lập trình động, giúp lập trình viên phát triển kỹ năng giải quyết bài toán hiệu quả và tiết kiệm tài nguyên.

  Làm việc với cấu trúc dữ liệu phức tạp: Đệ quy là một công cụ không thể thiếu khi làm việc với cây, đồ thị, hoặc các cấu trúc dữ liệu tổ chức theo hệ thống phân cấp.

  Tăng tư duy sáng tạo: Việc hiểu và triển khai đệ quy đòi hỏi lập trình viên suy nghĩ linh hoạt và sáng tạo, từ đó nâng cao khả năng tư duy trong lập trình.

  Dễ bảo trì và mở rộng mã nguồn: Các giải thuật được viết bằng đệ quy thường ngắn gọn, dễ đọc, giúp mã nguồn dễ bảo trì hơn khi làm việc trong các nhóm lập trình lớn.

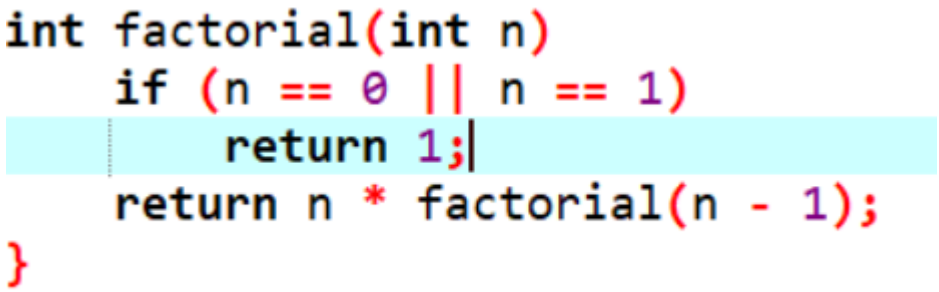
CHƯƠNG 2: PHÂN TÍCH CHUYÊN SÂU VỀ HÀM ĐỆ QUY

2.1 Các dạng đệ quy

Trong các bài trước, chúng ta đã nhắc lại khá chi tiết về đệ quy và làm việc với một số ví dụ rất điển hình về việc sử dụng đệ quy trong các bài toán thực tế. Tuy nhiên, trên thực tế có nhiều cách triển khai đệ quy khác nhau. Trong bài này, chúng ta sẽ tìm hiểu bốn dạng khác nhau của đệ quy: Đệ quy trực tiếp, đệ quy gián tiếp, đệ quy đuôi và đệ quy đầu.

Đệ quy trực tiếp

Một hàm được gọi là đệ quy trực tiếp nếu nó gọi đến chính nó một cách lặp đi lặp lại trong nội dung thân của hàm. Dạng đệ quy này phổ biến nhất, và ta cũng đã thấy ở các bài trước. Dưới đây là một ví dụ về đệ quy trực tiếp:



ảnh 2.1 cấu trúc của đệ quy trực tiếp

Trong chương trình này, bạn có một hàm tên là func gọi đến chính nó trong phần thân của hàm. Do đó, bạn có thể gọi nó là đệ quy trực tiếp.

- **Đệ quy gián tiếp**

  Dạng đệ quy trong đó hàm gọi chính nó thông qua việc gọi một hàm khác chứa nó được gọi là đệ quy gián tiếp. Một chương trình đệ quy gián tiếp có cấu trúc cơ bản như sau:

```
void func1(int z);
void func2(int y) {
    if (y <= 0) {
        return;
    }
    cout << "func2: " << y << endl;
    func1(y - 2);
}

void func1(int z) {
    if (z <= 0) {
        return;
    }
    cout << "func1: " << z << endl;
    func2(z - 1);
}
```

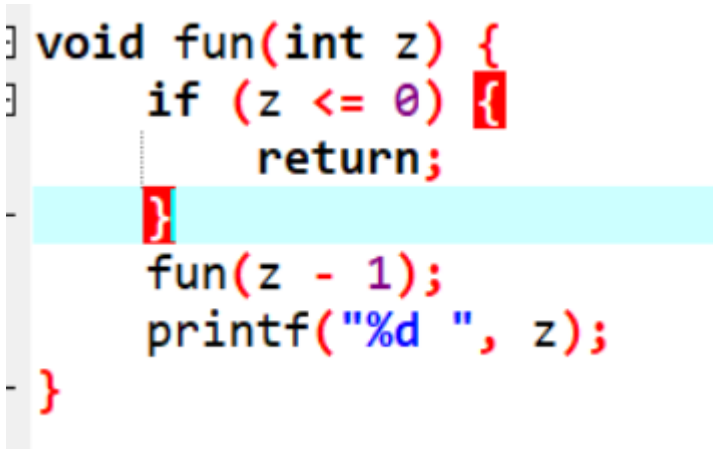
ảnh 2.2 ví dụ minh họa về đệ quy gián tiếp

Trong dạng này, ta có thể thấy hàm func1 gọi đến hàm func2, và đến lượt mình hàm func2 lại gọi lại hàm func1. Do đó, ta thường gọi dạng này là đệ quy gián tiếp.

Đệ quy đầu

Một hàm được gọi là đệ quy đầu nếu lời gọi đệ quy được thực hiện trước các thao tác khác trong hàm. Sau khi trả lại, sẽ có một cái gì đó cần đánh giá. Vì vậy, dạng đệ quy này về cơ bản khá là khó khăn để chuyển thể thành dạng cài đặt sử dụng vòng lặp.

Xét ví dụ sau:

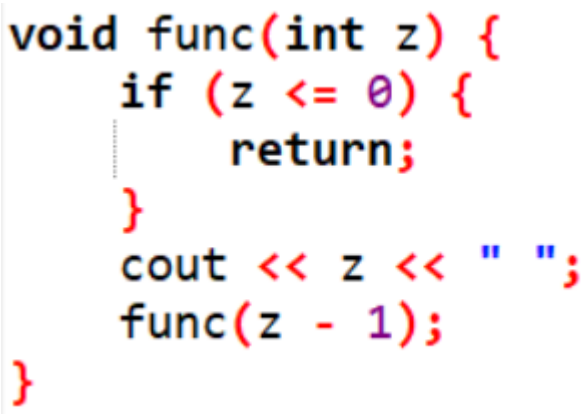


ảnh 2.3 ví dụ minh họa về đệ quy gián tiếp

Trong hàm này, bạn có thể thấy rằng có các thao tác khác cần được thực thi sau lời gọi đệ quy. Do đó, chương trình cần nhớ các trạng thái trước đó bên trong của khối lệnh này. Điều này gây khó khăn cho quá trình chuyển thể nó thành vòng lặp.

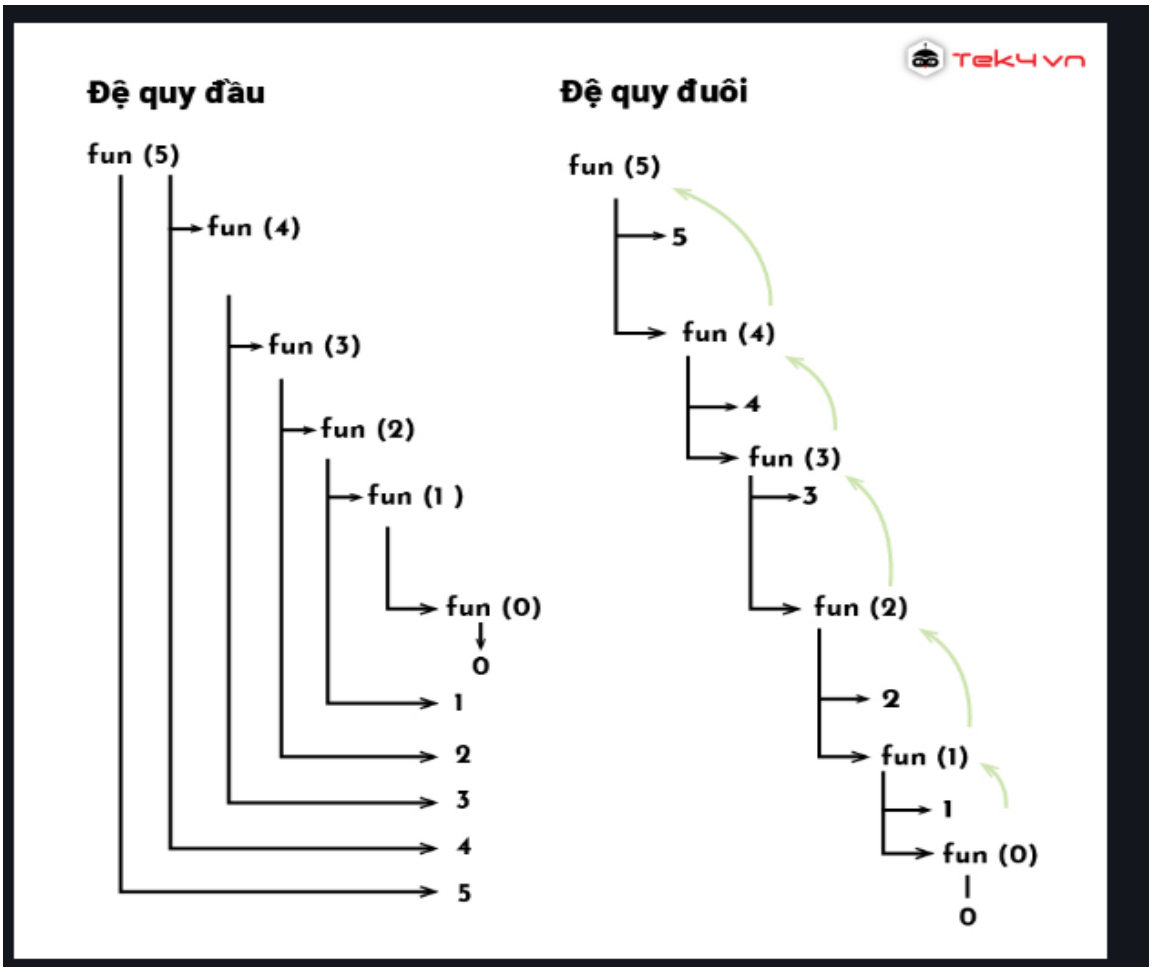
Nói chung, đệ quy đuôi luôn tốt hơn. Mặc dù cả hai đều có cùng độ phức tạp về thời gian và không gian, nhưng các dạng đệ quy đuôi có lợi thế hơn về bộ nhớ trong ngăn xếp của hàm. Các đệ quy đầu sẽ cần chờ đợi trong bộ nhớ ngăn xếp hàm cho đến khi các câu lệnh đệ quy được thực thi hoàn tất. Điều này gây ra độ trễ trong kết quả tổng thể, trong khi các đệ quy đuôi sẽ dừng ngay trong ngăn xếp hàm khi thực thi.

Đệ quy đuôi



ảnh 2.4 ví dụ minh họa về đệ quy đuôi

Một hàm đệ quy được gọi là đệ quy đuôi nếu như lời gọi đệ quy là hàm được gọi cuối cùng được thực hiện trong thân hàm đệ quy. Dưới đây là cấu trúc của một hàm đệ quy đuôi:



ảnh 2.5 cách thức hoạt động của đệ quy đuôi

Một điều nữa đó là chúng ta sẽ rất khó chuyển thể dạng đệ quy đầu sang các dạng sử dụng vòng lặp để tăng tốc độ thực thi cũng như vượt qua hạn chế về không gian bộ nhớ được cho phép bởi đệ quy bởi các trình biên dịch và hệ điều hành. Trên thực tế đệ quy trực tiếp và đệ quy đuôi là hai dạng phổ biến nhất thường gặp.

Tiếp theo, chúng ta hãy tiếp tục thực hành với một số bài tập để thành thạo các dạng đệ quy này.

2.2 Cấu tạo của hàm đệ quy

Hàm đệ quy có cấu tạo khá đơn giản và dễ sử dụng gồm 2 phần đó là:

trường hợp cơ bản base case

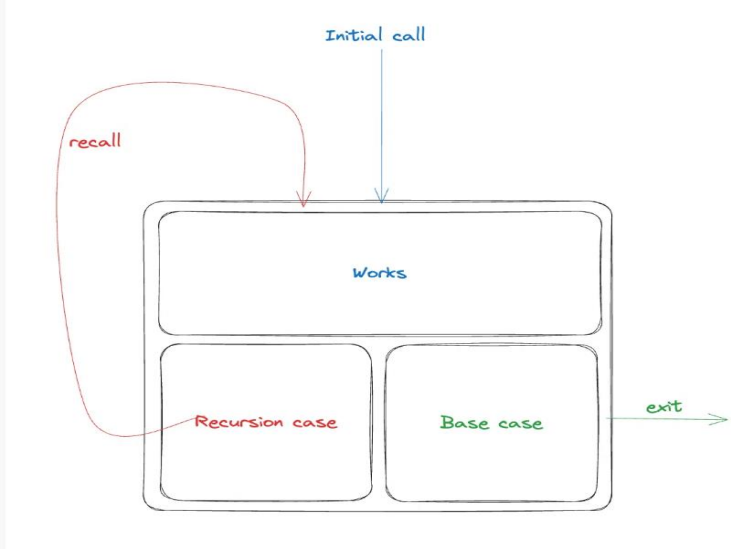
-Trường hợp cơ bản (base case) của đệ quy là thành phần nhỏ nhất trong đệ quy, là trường hợp mà bài toán đã được giải quyết mà không cần phải gọi lại chính nó. Bản chất cái tên "base" cũng đã ám chỉ việc nó là nền móng quan trọng nhất trong đệ quy, nơi mà các bài toán khác cần phải sử dụng đến.

Mục tiêu của base case là tìm một điểm dừng cho đệ quy, vì nếu không có điểm dừng này, đệ quy sẽ bị chạy đến vô hạn số lần

-Trường hợp đệ quy (recursive case) của đệ quy là thành phần phức tạp hơn base case, do đó, nó cần thực hiện liên tiếp các hành động gọi lại để tìm ra lời giải, cho đến khi gọi đến base case.

Việc recursive case gọi liên tiếp gọi lại chính nó nhằm tiến gần đến base case, đồng thời giải quyết một bài toán nhỏ hơn so với bài toán ban đầu, cho đến khi gọi đến base case.

Mục tiêu của recursive case là cố gắng tìm ra một bài toán nhẹ nhàng, dễ giải quyết hơn cho đến khi đạt đến điểm dừng là base case.



ảnh 2.6 cách thức hoạt động của hàm đệ quy

2.3 Các thành phần của một hàm đệ quy

Một hàm đệ quy luôn luôn bao gồm hai phần:

- ***Phần neo:*** Chính là lời giải cho bài toán cơ sở, cũng là phần thể hiện ***tính dừng*** của thuật toán. Khi hàm đệ quy tự gọi lại chính nó tới một thời điểm nhất định thì sẽ phải đạt được phần neo, bởi vì bài toán không thể phân tách ra mãi được mà phải đạt tới một bài toán cơ sở đã có sẵn kết quả. Công việc ở phần neo rất đơn giản, có thể giải trực tiếp mà không cần thông qua bài toán nhỏ hơn nào cả.
- ***Phần đệ quy:*** Trong trường hợp bài toán chưa thể giải được bằng phần neo, ta sẽ xác định các bài toán con và gọi đệ quy tới các bài toán con đó để giải chúng. Sau khi đã có lời giải của các bài toán con rồi, ta phối hợp chúng lại bằng ***công thức truy hồi*** để có được lời giải của bài toán ban đầu. Phần này thể hiện tính ***quy nạp*** của thuật toán.

2.4 một số loại đệ quy thường gặp

Đệ quy không chỉ có một loại mà nó còn được tùy biến để phù hợp với nhiều loại bài toán khác nhau mà không làm ảnh hướng tới tính chất hoặc biến đổi quá mức để khiến nó trở thành một hàm khác.

a.Đệ quy tuyến tính - Linear Recursion

-Định nghĩa Đệ quy tuyến tính

Đệ quy tuyến tính (Linear Recursion) là một kỹ thuật lập trình nơi một hàm gọi lại chính nó một cách trực tiếp, nhưng chỉ thực hiện một lần trong mỗi lần gọi.

Điều này tạo nên một "chuỗi" của các lời gọi hàm, với mỗi lời gọi hàm mới dựa trên kết quả của lời gọi trước đó.

- Cấu trúc Đệ quy tuyến tính

Cấu trúc của đệ quy tuyến tính là cấu trúc đơn giản nhất trong đệ quy, khi nó luôn thực hiện gọi lại chính nó cho mỗi lần thực hiện, cho đến khi tìm đến base case.

- Base Case: Điều kiện cơ bản để kết thúc quá trình đệ quy. Nếu không có base case, hàm sẽ tiếp tục gọi chính nó vô tận.
- Recursive Case: Phần của hàm nơi nó gọi lại chính nó, nhưng với một đối số nhỏ hơn hoặc đơn giản hơn.

-Phân tích BigO

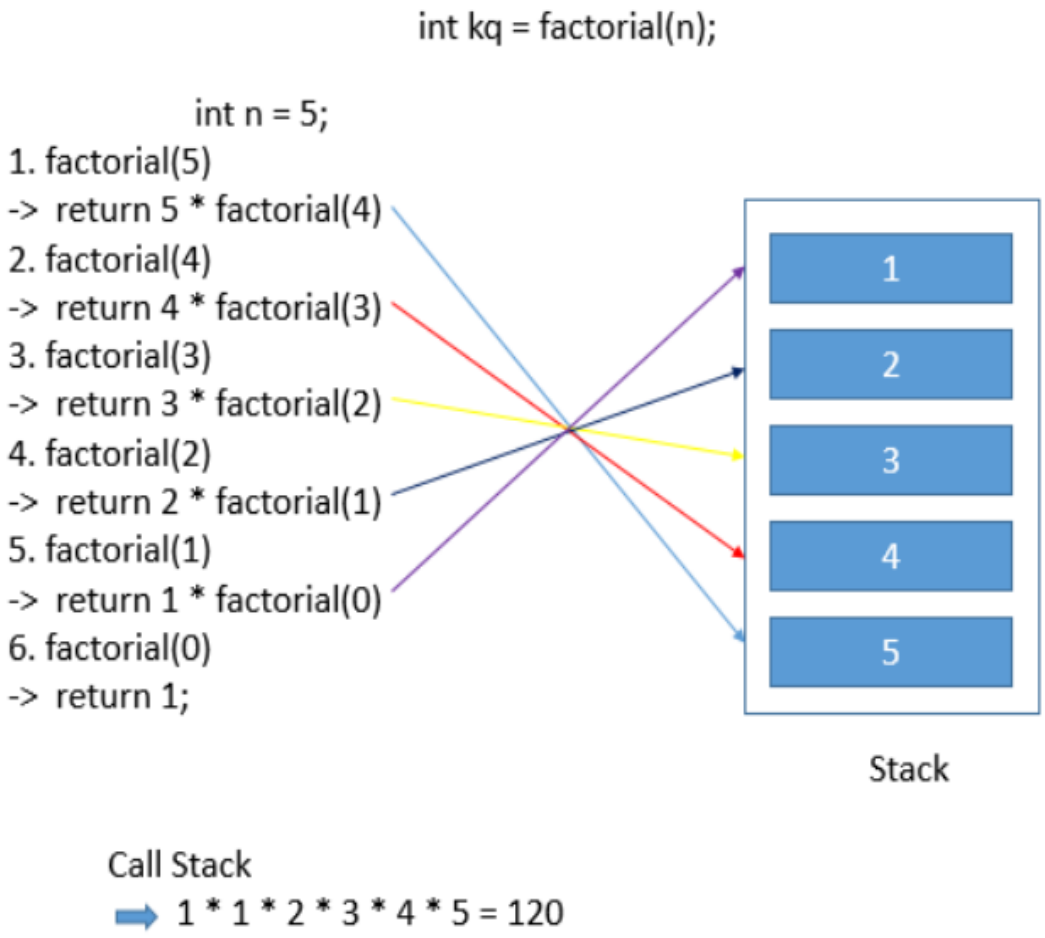
- Space Complexity: Trong đệ quy tuyến tính, độ phức tạp không gian (space complexity) thường là O(n) vì có n lời gọi hàm được lưu trữ trên stack.Time

```cpp
#include<iostream>
using namespace std;
int giaithua(int x){ //tao ham de quy
    if(x<0){ //dieu kien <0
        return 0; //tra ve 0
    }
    else if(x==0){ // neu x bang 0 ta tra ve 1
        return 1;
    }
    return x*giaithua(x-1); // tra ve gia tri x*giai thua
}
int main(){
    int a;
    cin>>a;
    cout<<giaithua(a); //goi ham de quy va in gia ham voi gia tri a
}
```

- ảnh 2.7. ví dụ về đệ quy tuyến tính

```
5
120
```

- Complexity: Độ phức tạp thời gian (time complexity) cũng là O(n) vì mỗi lần đệ quy, độ phức tạp về tính toán sẽ giảm đi một lượng cố định.



-

- ảnh 2.8 cách thức hoạt động của ví dụ trên

Giả sử nếu đệ quy tuyến tính được xác định và lập ra để lập bài toán giai thừa bằng 5! Thì hàm đệ quy sẽ chạy theo điều kiện từ trên xuống dưới. Kiểm tra bài toán nếu bằng 0 thì sẽ trả in ra giá trị 1, còn không sẽ liên tục gọi hàm không giá trị là 1 nhân với return +1 liên tục khi nào hàm factorial đạt số đã nhập trừ 1 thì dừng và in ra màn hình kết quả.

Hay như cách tính bình thường 1*1, 2*2,…

b Đệ quy nhị phân - Binary Recursion

- Định nghĩa Đệ quy nhị phân

Đệ quy nhị phân (Binary Recursion) là kỹ thuật lập trình mà trong đó mỗi hàm gọi lại chính nó hai lần trong mỗi lần thực thi.

Điều này làm cho kiểu đệ quy tạo có thể tạo ra một cây nhị phân tương đối phức tạp, với mỗi node trong cây có thể tạo ra hai node con tương ứng.

- Cấu trúc Đệ quy nhị phân

- Base Case: Điều kiện cơ bản để kết thúc quá trình đệ quy, tương đối giống với đệ quy tuyến tính.
- Recursive Case: Phần của hàm nơi nó gọi lại chính nó, nhưng không chỉ một, mà là hai lần, thường với đối số khác nhau.

- Phân tích BigO

- Space Complexity: Trong đệ quy nhị phân, độ phức tạp không gian (space complexity) thường là O(n), do kiểu đệ quy này lưu trữ một thông tin dưới dạng cây nhị phân
- Time Complexity: Độ phức tạp thời gian là O(2^n), biểu diễn cho số lượng lời gọi hàm tổng cộng mà ở đây là hai lần. Xin lưu ý rằng time complexity trên là cực kỳ lớn, do đó rất hiếm trường hợp mà chúng ta sử dụng kiểu đệ quy này trong thực tế.

Để hiểu về cách thức hoạt động của đệ quy nhị phân ta lấy ví dụ về giải bài toán tìm số Fibonanci dưới đây:

Dãy số Fibonacci là một chuỗi các số tự nhiên bắt đầu bằng 0 và 1, trong đó mỗi số tiếp theo bằng tổng của hai số trước đó. Dãy số này được đặt tên theo Leonardo Fibonacci, 2. Dãy Fibonacci là dãy vô hạn các số tự nhiên bắt đầu bằng 1 và 1, sau đó các số tiếp theo sẽ bằng tổng của 2 số liền trước nó

```cpp
#include<iostream>
using namespace std;
int Fibonacci(int x){ //day la khai bao cua ham de quy co ten la Fibonacci nhan doi so là so nguyên x
    if(x<=1){  //dieu kien cua ham de quy nho hon hoac bang 1
        return x; // tra ve x
    }
    return Fibonacci(x-1) + Fibonacci(x-2); //ham so tra ve tong cua Fibonacci(x - 1) và Fibonacci(x
}
int main(){
    int a;
    cin>>a; //nhap a
    cout<<Fibonacci(a); //goi va in ra ham da viet o truoc ham main
}
```

```
8
21
```

ảnh 2.9 ví dụ minh họa về đệ quy nhị phân

Từ tính chất của dãy số Fibonacci ta có thể hiểu được cách tính của hàm đệ quy như sau

Khi ta nhập một số vào, hàm sẽ liên tục tính từ giá trị Fibonacci bằng 2 sẽ có giá trị là 1+0 (2-1=1) (2-2=0) rồi ta sẽ lấy kết quả của phép tính trước rồi thay vào số thứ nhất rồi lấy số thứ 1 của phép tính trước rồi cộng với kết quả của phép tính thức 1 sẽ

Ngoài ra, các kiểu đệ quy đa nhánh (đệ quy thực hiện nhiều hơn hai lần cho mỗi lần gọi lại) cũng có thể phân tích như đệ quy nhị phân với time complexity thông thường là O(n^x) với n là số lần thực hiện đệ quy cho mỗi lần gọi lại.

c. Đệ quy phi tuyến

- Định nghĩa Đệ quy phi tuyến

Trong đệ quy phi tuyến, số lần một hàm gọi lại chính nó có thể thay đổi và không cố định, nó có thể phụ thuộc vào nhiều yếu tố khác nhau như giá trị đầu vào, điều kiện trong hàm, v.v.

Điều này tạo nên một cấu trúc phức tạp hơn so với đệ quy tuyến tính và nhị phân, nơi chúng ta có thể dự đoán trước được số lần gọi đệ quy ở mỗi bước.

- Cấu trúc Đệ quy phi tuyến

- Base Case: Điều kiện cơ bản để kết thúc quá trình đệ quy, tương đối giống với đệ quy tuyến tính.
- Recursive Case: Phần của hàm nơi nó gọi lại chính nó
  Đối với trường hợp đệ quy phi tuyến, các trường hợp gọi lại tương đối phức tạp và khó đoán, mỗi lần gọi hàm sẽ thực hiện một số lượng không ổn định, tạo ra một cấu trúc cây tương đối phức tạp cả về chiền rộng lẫn chiều sâu.

- Phân tích BigO

- Space Complexity: Trong đệ quy phi tuyến, độ phức tạp không gian (space complexity) thường là O(n), do kiểu đệ quy này lưu trữ một thông tin dưới dạng cây.
  Tuy nhiên, vẫn có các trường hợp khác do nó còn phụ thuộc lớn và chiều sâu của cây và bài toán cần giải quyết.
- Time Complexity: Vì mỗi lần gọi đệ quy, số lần thực hiện sẽ có độ biến thiên, vì thế độ phức tạp thời gian là sẽ được biểu diễn ở dạng O(x^n), nhưng còn phụ thuộc rất nhiều vào cách mà chúng ta xử lý ở mỗi lần gọi đệ quy.

d. Đệ quy lồng - Nested Recursion

- Định nghĩa Đệ quy lồng

Đệ quy lồng (Nested Recursion) là một dạng đệ quy khi lời gọi đệ quy thực hiện với một tham số là kết quả của một lời gọi đệ quy khác.

Điều này tạo nên một cấu trúc đệ quy "lồng vào nhau" hoặc "đệ quy trong đệ quy", khiến cho việc phân tích độ phức tạp cũng khó khăn hơn.

- Cấu trúc Đệ quy lồng

- Base Case: Điều kiện cơ bản để kết thúc quá trình đệ quy, tương đối giống với đệ quy tuyến tính. Đây là điều kiện là hàm gọi sẽ trả kết quả thay vì gọi một lồng đệ quy khác.

- Recursive Case: Phần của hàm nơi nó gọi lại chính nó

  Trong trường hợp đệ quy, đệ quy lồng thể hiện sự phức tạp thông qua việc một lời gọi đệ quy có thể chứa một hoặc nhiều lời gọi đệ quy khác như một đối số.

- Phân tích BigO

Do tính chất phức tạp tương tự với đệ quy phi tuyến, độ phức tạp thuật toán trong extra space và time complexity giống với đệ quy phi tuyến.

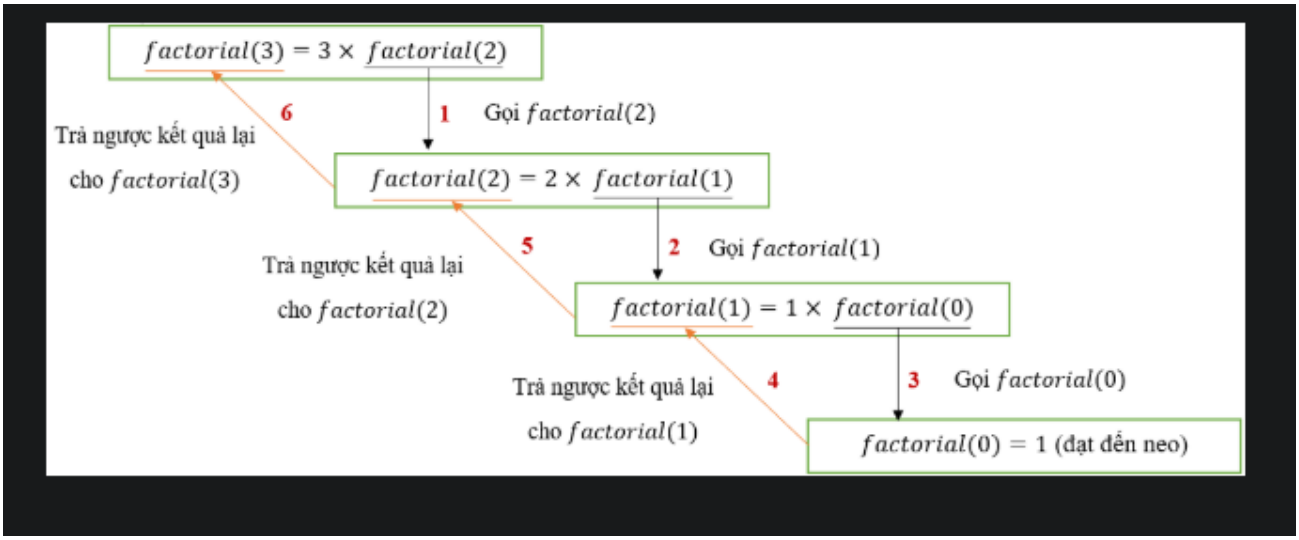
2.5 Cơ chế hoạt động của một hàm đệ quy

Ta lấy một ví dụ là hàm đệ quy tính n!n! như sau:

```
int factorial(int n)
{
    if (n == 0) // Phần neo.
    return 1;
    else // Phần gọi đệ quy.
    return factorial(n - 1) * n;
}
```

ảnh 2.10 ví dụ minh họa về cơ chế hoạt động của hàm đệ quy

Ở bài toán này, phần neo định nghĩa trước lời giải cho trường hợp n=0 là ,1, còn đối với các bài toán có n>1, ta sẽ dùng lời gọi đệ quy để đưa về giải bài toán có kích thước bằng n−1 rồi từ đó tính n!=(n−1)!×n. Chẳng hạn, nếu dùng hàm này để tính 3!, giải thuật sẽ diễn ra như sau:



ảnh 2.11 cách thức hoạt động theo sơ đồ

Việc giải bài toán factorial(3)*factorial*(3) sẽ diễn ra trong 66 từ việc phân rã factorial(3)*factorial*(3) xuống factorial(0)*factorial*(0) rồi lần lượt trả ngược kết quả các bài toán nhỏ lên các bài toán lớn hơn đã gọi nó, cho tới khi đạt được kết quả của bài toán ban đầu. Nguyên lí của việc này là do máy tính khi nhận thấy một bài toán chưa được giải ở lời gọi đệ quy, nó sẽ tạm thời lưu bài toán đó vào một ***ngăn xếp*** theo chiều từ trên xuống dưới, như vậy các bài toán chưa được giải sẽ xếp chồng lên nhau theo thứ tự bài toán lớn hơn ở dưới, bài toán nhỏ hơn ở trên. Việc giải các bài toán lại được thực hiện từ trên xuống dưới, như vậy bài toán ở bên dưới (là bài toán lớn hơn) sẽ thu nhận được kết quả của bài toán bên trên (là bài toán nhỏ hơn),...Tiếp tục thực hiện như vậy cho tới khi trong ***ngăn xếp*** chỉ còn lại một bài toán cuối cùng, đó chính là bài toán gốc.

Đây là một đặc điểm rất thú vị của đệ quy, có ứng dụng rất lớn trong việc thiết kế các giải thuật sau này như: Quay lui, nhánh cận,...

2.7 So sánh đệ quy với các phương pháp lập trình khác

So sánh đệ quy với các phương pháp lập trình khác giúp ta hiểu rõ hơn về ưu và nhược điểm của từng phương pháp. Dưới đây là một so sánh chi tiết giữa đệ quy và các phương pháp lập trình khác:

1. Đệ quy (Recursion)

Ưu điểm:

- Mã nguồn thường ngắn gọn, dễ hiểu và thể hiện rõ cấu trúc của bài toán đệ quy tự nhiên.
- Thích hợp cho các bài toán có cấu trúc đệ quy như duyệt cây, đồ thị, và phân chia để trị (divide and conquer).

Nhược điểm:

- Có thể gây lỗi tràn bộ nhớ (stack overflow) nếu không có điều kiện dừng đúng.
- Tốn nhiều tài nguyên bộ nhớ hơn so với phương pháp lặp do tạo ra nhiều khung ngăn xếp (stack frames).

2. Lặp (Iteration)

Ưu điểm:

- Hiệu suất bộ nhớ tốt hơn vì không cần tạo ra nhiều khung ngăn xếp.
- Thường dễ kiểm soát và hiểu hơn đối với người mới học lập trình.

Nhược điểm:

- Mã nguồn có thể dài hơn và khó đọc nếu bài toán có cấu trúc phức tạp.
- Không thích hợp cho các bài toán có cấu trúc đệ quy tự nhiên.

3. Chia để trị (Divide and Conquer)

Đây là một phương pháp chia bài toán lớn thành các bài toán nhỏ hơn, giải quyết từng phần và kết hợp kết quả lại.

Ưu điểm:

- Hiệu quả cho các bài toán lớn và phức tạp, giảm độ phức tạp tính toán.
- Dễ dàng áp dụng cho các bài toán có cấu trúc phân chia rõ ràng.

Nhược điểm:

- Đôi khi khó xác định cách phân chia bài toán hợp lý.

- Có thể tốn nhiều tài nguyên nếu không tối ưu hóa.

4. Quy hoạch động (Dynamic Programming)

Quy hoạch động giải quyết bài toán bằng cách lưu trữ kết quả của các phép tính đã thực hiện, tránh tính toán lại nhiều lần.

Ưu điểm:

- Hiệu quả trong việc giải quyết các bài toán có tính chất lặp lại nhiều lần.
- Giảm độ phức tạp tính toán bằng cách lưu trữ và tái sử dụng kết quả.

Nhược điểm:

- Đòi hỏi bộ nhớ để lưu trữ các kết quả trung gian.
- Cần có kỹ năng thiết kế thuật toán để áp dụng đúng cách.

5. Lập trình hàm (Functional Programming)

Đây là một phong cách lập trình dựa trên các hàm toán học, không có trạng thái và biến đổi dữ liệu.

Ưu điểm:

- Dễ bảo trì và kiểm tra vì không có trạng thái biến đổi.
- Thích hợp cho các bài toán có tính chất toán học và cần xử lý dữ liệu mạnh mẽ.

Nhược điểm:

- Đôi khi khó hiểu và áp dụng cho người mới học lập trình.
- Có thể không hiệu quả cho các bài toán cần tối ưu hóa bộ nhớ và hiệu suất.

CHƯƠNG 3: ĐỆ QUY TRONG THỰC TẾ

Hàm đệ quy là một trong những hàm được sử dụng thông dụng trong lập trình dù có là ngôn ngữ lập trinh nào đi chăng nữa. Từ tinh phổ biến đó, đệ quy đã góp phần không nhỏ giúp các lập trình viên phát triển các chương trinh, phần mềm của minh một cách trơn chu hơn.

Trong chương 3 này, ta sẽ được công dụng của hàm đệ quy trong những chương trình, các bài toán phổ biến khi sử dụng hàm đệ quy và một số ứng dụng khác trong nhanh lập trinh của hàm đệ quy.

3.1 MỘT SỐ BÀI TOÁN ĐỆ QUY

Sinh các dãy nhị phân

Sinh tổ hợp (tập hợp con)

Đây là 2 bài toán ta sử dụng hàm đệ quy để giải quyết. Để giải quyết 2 bài toán này ta phải đáp ứng đủ những tiêu chí sau đây.

Phải có đủ hàm base case và recursive case cho bài toán

Điều kiện dừng:

Mỗi bài toán đều cần điều kiện rõ ràng để kết thúc đệ quy. Nếu không, chương trình sẽ gặp lỗi stack overflow (ngăn xếp bị tràn).

Xử lý trạng thái:

Tại mỗi bước trong đệ quy, lưu và kiểm tra trạng thái hiện tại (ví dụ: bàn cờ, dãy nhị phân, tập hợp con).

Quay lui (Backtracking):

Sau mỗi lời gọi đệ quy, cần "quay lui" để khôi phục trạng thái ban đầu (đặc biệt quan trọng trong bài toán tổ hợp và xếp hậu)

Dựa trên những điều kiện này, hãy bắt đầu vào áp dụng nó vào chương trình.

3.1.1 bài toán sinh các dãy nhị phân

Để vận hành một hàm đệ quy, chúng ta cần tạo ra một hàm có thể tạo ra các giá trị nhị phân có độ dài là một số nguyên. Tiếp đó là một hàm đệ quy để chuyển giá trị thập phân sang giá trị nhị phân.

```
void generateBinary(int n, int arr[], int i) {
    if (i == n) {
        for (int j = 0; j < n; j++) {
            cout << arr[j];
        }
        cout << endl;
        return;
    }

    arr[i] = 0;
    generateBinary(n, arr, i + 1);
    arr[i] = 1;
    generateBinary(n, arr, i + 1);
}
```

ảnh 3.1 phân tích hàm đệ quy của bài toán sinh các dãy nhị phân

Ở đây ta có hàm "generateBinary" để tạo ra hàm đệ quy với "n", "j" là các giá trị số thực, và mảng "arr".

Điều kiện là nếu i bằng giá trị n, tạo ra một vòng lặp với giá trị ban đầu "j" bằng 0, "j" nhỏ hơn n, và "j" luôn tăng. In ra mảng "arr" với giá trị "j".

Khi ra khỏi vòng lặp, chuyển giá trị của mảng i thành các giá trị thập phân. Với các giá trị là 0 và 1.

```
int main() {
    int n;
    cout << "Nhap do dai day nhi phan: ";
    cin >> n;

    int arr[n];
    cout << "Cac day nhi phan co do dai " << n << " la: " << endl;
    generateBinary(n, arr, 0);

    return 0;
}
```

3.2 code hàm main của bài toán sinh các dãy nhị phân

```
#include <iostream>
using namespace std;
void generateBinary(int n, int arr[], int i) {
    if (i == n) {
        for (int j = 0; j < n; j++) {
            cout << arr[j];
        }
        cout << endl;
        return;
    }

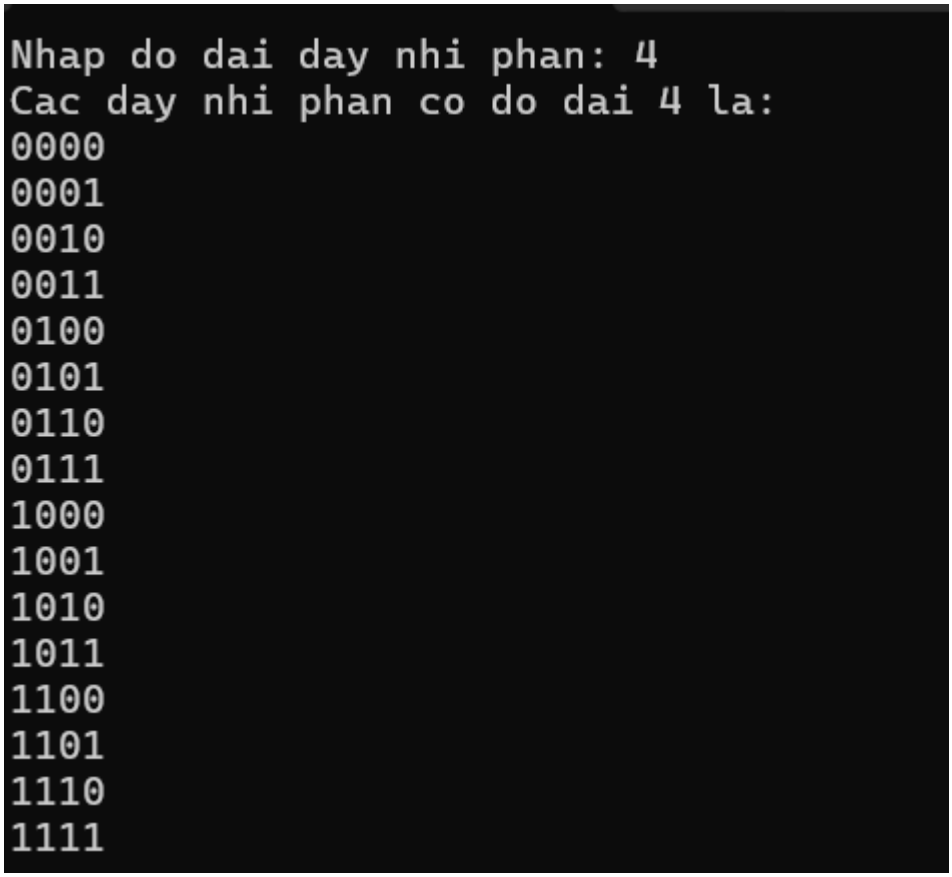
    arr[i] = 0;
    generateBinary(n, arr, i + 1);
    arr[i] = 1;
    generateBinary(n, arr, i + 1);
}
int main() {
    int n;
    cout << "Nhap do dai day nhi phan: ";
    cin >> n;

    int arr[n];
    cout << "Cac day nhi phan co do dai " << n << " la: " << endl;
    generateBinary(n, arr, 0);

    return 0;
}
```

3.3 code toàn bộ bài toán sinh các dãy nhị phân

khi xong phần hàm đệ quy, ta chuyển xuống hàm “main” để in và nhập vào các giá trị để rồi gọi lại hàm “generateBinary” để chuyển về giá trị nhị phân



ảnh 3.4 kết quả của bài toán sinh các dãy nhị phân

Khi nhập giá trị 4 từ bàn phím, máy tính sẽ cho ra các biến đổi để được giá trị 4 trong hệ thập phân là “1111”

3.1.2 Sinh tổ hợp (tập hợp con)

Mục tiêu của bài toán này là sinh ra các tổ hợp con của một số đã được nhập từ bàn phím.

```
void generateSubsets(int set[], int subset[], int n, int index, int subsetSize) {
    if (index == n) {
        cout << "{ ";
        for (int i = 0; i < subsetSize; i++) {
            cout << subset[i] << " ";
        }
        cout << "}" << endl;
        return;
    }
    generateSubsets(set, subset, n, index + 1, subsetSize);

    subset[subsetSize] = set[index];
    generateSubsets(set, subset, n, index + 1, subsetSize + 1);
}
```

ảnh 3.5 code hàm đệ của bài toán sinh tổ hợp

set: Mảng chứa tập hợp ban đầu, gồm n phần tử.

subset: Mảng tạm thời để lưu các phần tử hiện tại của tập con.

n: Kích thước của tập hợp ban đầu.

index: Vị trí của phần tử hiện tại trong tập hợp.

subsetSize: Số lượng phần tử hiện tại trong tập con

Khi index == n (đã duyệt hết các phần tử trong tập hợp ban đầu), hàm sẽ:

In tập hợp con hiện tại được lưu trong subset.

Kết thúc lời gọi đệ quy bằng return

Hàm đệ quy được gọi để bỏ qua phần tử tại vị trí index và tiếp tục xử lý phần tử kế tiếp.

subsetSize không tăng vì không thêm phần tử vào subset

Gọi đệ quy khi chọn phần tử tại vị trí index**:**

- Thêm phần tử set[index] vào subset.
- Tăng kích thước của tập con (subsetSize + 1).
- Gọi đệ quy để xử lý các phần tử tiếp theo.

```
int main() {
    int n;
    cout << "Nhap so luong phan tu cua tap hop: ";
    cin >> n;

    int set[n];
    cout << "Nhap cac phan tu cua tap hop: ";
    for (int i = 0; i < n; i++) {
        cin >> set[i];
    }
    int subset[n];
    cout << "Cac tap hop con la:" << endl;
    generateSubsets(set, subset, n, 0, 0);
    return 0;
}
```

3.6 code hàm main của bài toán sinh tổ hợp

```
Nhap so luong phan tu cua tap hop: 3
Nhap cac phan tu cua tap hop: 6
7
2
Cac tap hop con la:
{ }
{ 2 }
{ 7 }
{ 7 2 }
{ 6 }
{ 6 2 }
{ 6 7 }
{ 6 7 2 }
```

3.7 kết quả của bài toán sinh tổ hợp

Khi nhập số lượng phần tử, và các phần tử trong tập hợp hàm sẽ sắp xếp lại phần tử sao cho chúng có không còn có thể sắp xếp nữa thì sẽ in ra số phần từ ta đã nhập từ bàn phím theo trình tự.

3.2 những ứng dụng thực tế của đệ quy

Đệ quy không chỉ là một khái niệm lý thuyết; nó có rất nhiều ứng dụng thực tế trong cuộc sống và trong công nghệ. Dưới đây là một số ví dụ nổi bật:

1. **Duyệt cây và đồ thị**: Đệ quy thường được sử dụng để duyệt các cấu trúc dữ liệu cây và đồ thị. Các thuật toán như tìm kiếm theo chiều sâu (DFS) và tìm kiếm theo chiều rộng (BFS) đều có thể được triển khai bằng cách sử dụng đệ quy.
2. **Sắp xếp**: Các thuật toán sắp xếp nổi tiếng như sắp xếp nhanh (quicksort) và sắp xếp trộn (mergesort) đều dựa vào đệ quy để phân chia và xử lý các mảng dữ liệu.
3. **Tính toán các dãy số**: Đệ quy rất hữu ích trong việc tính toán các dãy số, chẳng hạn như dãy Fibonacci. Ví dụ, dãy Fibonacci có thể được tính bằng một hàm đệ quy đơn giản.
4. **Giải bài toán đệ quy**: Nhiều bài toán có tính chất đệ quy tự nhiên, chẳng hạn như bài toán tháp Hà Nội, bài toán n-queens, và bài toán đi qua mê cung, đều có thể được giải quyết bằng đệ quy.

Ví dụ về cây thư mục:

Trong quản lý hệ thống tập tin, đệ quy được sử dụng để duyệt các thư mục và tệp tin trong một hệ thống cây thư mục. Hàm duyệt cây thư mục sẽ gọi lại chính nó để xử lý từng thư mục con cho đến khi không còn thư mục con nào.

3.3 Những lỗi thường gặp khi dung đệ quy

Khi sử dụng đệ quy, lập trình viên có thể gặp phải một số lỗi phổ biến. Dưới đây là một số lỗi thường gặp và cách khắc phục chúng:

1. Thiếu điều kiện dừng hoặc điều kiện dừng sai

Nếu không có điều kiện dừng hoặc điều kiện dừng không chính xác, hàm đệ quy sẽ gọi lại chính nó vô hạn và gây lỗi tràn bộ nhớ (stack overflow).

Cách khắc phục: Luôn đảm bảo rằng hàm đệ quy có điều kiện dừng rõ ràng và chính xác.

2. Lời gọi đệ quy không tiến dần đến điều kiện dừng

Nếu tham số truyền vào trong lời gọi đệ quy không thay đổi hoặc không tiến dần đến điều kiện dừng, hàm cũng sẽ gây ra lỗi đệ quy vô hạn.

Cách khắc phục: Đảm bảo rằng mỗi lần gọi đệ quy, tham số truyền vào thay đổi một cách hợp lý để tiến đến điều kiện dừng.

3. Lãng phí bộ nhớ

Hàm đệ quy thường tạo ra nhiều khung ngăn xếp (stack frames), dẫn đến lãng phí bộ nhớ và giảm hiệu suất nếu không được tối ưu hóa.

Cách khắc phục: Sử dụng đệ quy đuôi (tail recursion) nếu ngôn ngữ lập trình hỗ trợ, hoặc chuyển sang sử dụng vòng lặp để giảm thiểu sử dụng bộ nhớ.

4. Hiệu suất kém

Một số bài toán đệ quy có thể bị lặp lại nhiều lần, dẫn đến hiệu suất kém. Ví dụ, tính dãy Fibonacci bằng đệ quy đơn giản sẽ lặp lại nhiều lần cùng một phép tính.

Cách khắc phục: Sử dụng kỹ thuật nhớ (memoization) để lưu trữ kết quả của các lời gọi đệ quy đã thực hiện.

5. Gây khó hiểu và khó bảo trì mã nguồn

Mã nguồn sử dụng đệ quy có thể khó đọc và hiểu hơn so với mã sử dụng vòng lặp, đặc biệt đối với những người mới học lập trình.

Cách khắc phục: Luôn viết chú thích rõ ràng và giải thích logic của hàm đệ quy. Nếu mã nguồn quá phức tạp, hãy xem xét sử dụng vòng lặp thay vì đệ quy.

KẾT LUẬN

Qua 3 chương vừa đã được đề cập ở trên, có thể đưa ra kết luật về hệ đệ quy trong ngôn ngữ lập trình như sau:

Về ưu điểm: Tạo tư lập trình:

- Đệ quy giúp diễn đạt bài toán một cách trực quan và dễ hiểu. Các bài toán như xếp hậu, duyệt cây, hoặc Fibonacci thường được biểu diễn đệ quy dễ dàng hơn vì giải pháp của chúng có thể được chia nhỏ thành các bài toán con tương tự.

Mã ngắn gọn :

- Sử dụng đệ quy thường giúp giảm độ dài mã nguồn so với các phương pháp lặp (iteration). Điều này giúp chương trình dễ đọc, dễ bảo trì và ít lỗi logic.

Phù hợp với bài toán phân nhánh:

- Đệ quy rất mạnh mẽ khi xử lý các bài toán phân nhánh như duyệt cây, đồ thị (DFS), hay xử lý các bài toán chia để trị (divide and conquer) như quicksort hoặc mergesort.

Tự động lưu trữ trạng thái:

- Trong đệ quy, ngăn xếp của chương trình tự động lưu trữ trạng thái của các lời gọi trước đó, không cần phải tạo thêm các cấu trúc dữ liệu như mảng hoặc ngăn xếp thủ công.

Hiệu quả trong một số thuật toán:

- Một số thuật toán như tìm kiếm nhị phân, quay lui (backtracking), hay chia để trị đạt hiệu quả cao khi sử dụng đệ quy

Về nhược điểm: tuy có ưu điểm như vậy, nhưng hệ đệ quy lại có các nhược điểm không thể tránh khỏi như:

Dễ gây tràn ngăn xếp (Stack Overflow):

- Với các bài toán có số lượng lời gọi đệ quy lớn hoặc độ sâu đệ quy lớn (như n rất lớn), ngăn xếp của chương trình có thể bị tràn (stack overflow), dẫn đến lỗi chương trình.

Hiệu suất không tối ưu:

- Đệ quy thường tốn tài nguyên hơn vòng lặp do mỗi lời gọi hàm đệ quy sẽ tiêu tốn bộ nhớ và thời gian để lưu trữ trạng thái. Đặc biệt, nếu không tối ưu (như không sử dụng ghi nhớ - memoization), đệ quy có thể bị lặp lại những phép tính giống nhau nhiều lần.

Khó kiểm soát lỗi:

- Đệ quy yêu cầu điểm dừng (base case) rõ ràng. Nếu không xác định đúng điểm dừng hoặc logic bị lỗi, chương trình dễ rơi vào vòng lặp vô tận, gây tràn ngăn xếp.

Khó hiểu đối với người mới học:

- Đối với người mới bắt đầu, tư duy và viết đệ quy có thể khó hơn vòng lặp thông thường, đặc biệt với các bài toán phức tạp.

Phụ thuộc vào hệ thống:

- Khả năng xử lý đệ quy phụ thuộc vào ngăn xếp gọi hàm (call stack) của hệ thống. Một số ngôn ngữ hoặc trình biên dịch có giới hạn độ sâu đệ quy, dẫn đến hạn chế trong việc áp dụng với dữ liệu lớn.

Khó tối ưu hóa với một số bài toán:

- Với các bài toán yêu cầu hiệu suất cao như tìm kiếm hoặc xử lý trên tập dữ liệu lớn, viết bằng vòng lặp có thể nhanh và tối ưu hơn

Hướng phát triển của giải thuật đệ quy: • **Tối ưu hóa đệ quy**:

- Kết hợp đệ quy với các kỹ thuật như **Memoization** hoặc **Dynamic Programming** để tránh việc lặp lại các tính toán giống nhau, giúp cải thiện hiệu suất.
- Ví dụ: Bài toán dãy Fibonacci có thể được tối ưu bằng cách lưu trữ giá trị đã tính toán.

- **Tận dụng đệ quy đuôi (Tail Recursion)**:

- Phát triển và áp dụng đệ quy đuôi để giảm thiểu việc sử dụng ngăn xếp trong hệ thống, nhờ khả năng chuyển đổi thành vòng lặp hiệu quả hơn.

Áp dụng trong lĩnh vực hiện đại:

- Đệ quy có thể được áp dụng trong các thuật toán trí tuệ nhân tạo, học máy (machine learning) để xử lý cây quyết định (Decision Tree) hoặc hồi quy (Regression).
- Phát triển các thuật toán đệ quy trong đồ họa máy tính, ví dụ như vẽ Fractal hay Ray Tracing.

Nâng cao tính thích nghi của đệ quy:

- Tích hợp đệ quy với các kỹ thuật tính toán song song (Parallel Computing) để xử lý dữ liệu lớn hoặc bài toán phức tạp một cách nhanh chóng hơn.

Ứng dụng trong phân tích dữ liệu và tối ưu hóa:

- Phát triển thuật toán đệ quy để tìm kiếm lời giải tối ưu trong các bài toán tổ hợp hoặc tối ưu hóa, như cây bao trùm tối thiểu (Minimum Spanning Tree) hay bài toán lộ trình ngắn nhất (Shortest Path).

Khám phá đệ quy trong các mô hình toán học:

- Sử dụng đệ quy để mô phỏng hoặc giải các bài toán trong lĩnh vực toán học và khoa học tự nhiên, ví dụ: Phương pháp Monte Carlo, chuỗi Fibonacci trong sinh học.

Phân công công việc các thành viên trong nhóm

1. ***Nguyễn Chung Thành : 30% đóng góp***

-viết word báo báo

-tìm tài liệu

2. ***Đặng Quang Tiến : 35%***

- Kiểm thử phần mềm
- Chạy demo code
- Fix code

3. ***Phạm Anh Quân: 35%***

- Tìm tài liệu
- Viết code phần đầu
- Xem lỗi

DANH MỤC TÀI LIỆU THAM KHẢO

[1]. Giảng viên: TS. Trần Quý Nan CẤU TRÚC DỮ LIỆU & GIẢI THUẬT ĐỆ QUY VÀ QUAY LUI

[2]. Lương Việt Hải trên trang https://200lab.io/blog/de-quy-la-gi

[3]. Trên trang https://freetuts.net/giai-thuat/giai-thuat-de-quy

[4]. Phạm Quốc Hùng (2017), *Đề cương bài giảng Mạng máy tính*, Đại học SPKT Hưng Yên.